
† **Problem 0.0.1** (*Das Problem des Handlungsreisenden/Traveling Salesman Problem*) Given n cities, with the distance between them. Then, we want to find the shortest cycle, which goes through each cities.

Solution 1: With a start point, then at most $\frac{(n-1)!}{2}$ possibilities.

† **Problem 0.0.2** (*Calculation of Pressure of Network*) The calculation of graph of networks

Chapter 1 Fundamentals

1.0.1 Programs

The following list:

Table 1.1: Struktur und Bestandteile der Programmiersprache

Einheit	Bestandteile
Block	{ Programm }
Funktion	Funktionskopf Block
Funktionskopf	Rückgabetyt Funktionsname (Argumente) ; Ausdruck ;
Anweisung	if-Bedingung while-Schleife for-Schleife Block
Ausdruck	Konstante Variablenauswertung Funktionsaufruf Operatoranwendung (Ausdrücke)
Programm	Variablendeklarationen, Funktionen, Anweisungen

1.1 Logic

Omitted.

$$NAND(a, b) := \neg(a \wedge b) = \neg a \vee \neg b$$

$$Nor(a, b) := \neg(a \vee b) = \neg a \wedge \neg b$$

$$Xor(a, b) := a \oplus b := a \neq b$$

With C-codes:

Table 1.2: Logische Operatoren und Syntaxvergleich

Logik	Schriftform	C-Syntax	Assoziativität
Negation	$\neg$	!	von rechts nach links
Gleich	=	==	von links nach rechts
Ungleich	$\neq$	!=	von links nach rechts
Und	$\wedge$	&&	von links nach rechts
Oder	$\vee$		von links nach rechts
Zuweisung	$\leftarrow$	=	von rechts nach links

Definition 1.1.0.1 ...

Definition 1.1.0.2 The output is logically false, iff it is 0.
The definition is well-defined for the output with image except 0 and 1.

Definition 1.1.0.3 A **program** consists of a sequence of commands.
The most simple command is ;.
And

1.1.1 If-Else

1.2 Set

Omitted

1.3 Bit Operations

Definition 1.3.0.1 A **bit** is the basic unit of information.

And:

Definition 1.3.0.2 Define the

Especially:

Definition 1.3.0.3 A word is a sequence of bits: With length n called n -bit.

- A **byte** is a 8-bit;
- A **nibble** is a 4-bit;

1.3.1 Logic Operations(for words)

$$\begin{aligned}\sim w &:= (\neg w_{n-1}, \dots, \neg w_0) \\ v \&w &:= (v_{n-1} \wedge w_{n-1}, \dots, v_0 \wedge w_0) \\ v | w &:= (v_{n-1} \vee w_{n-1}, \dots, v_0 \vee w_0) \\ v \oplus w &:= (v_{n-1} \oplus w_{n-1}, \dots, v_0 \oplus w_0)\end{aligned}$$

Remark In C-program the "int" is not universal: (For example) It depends on `#include <stdint.h>`.

Definition 1.3.1.1 (Logic Shifting) Define the operation:

$$\begin{aligned}w \ll m &:= (w_{n-1-m}, \dots, w_0, 0, \dots, 0) \\ w \gg m &:= (0, \dots, 0, w_{n-1}, \dots, w_m)\end{aligned}$$

Especially, $m = 0, w \ll (-) = id, w \gg (-) = id$.

Definition 1.3.1.2 (0-Word) A 0-word is defined as the word with all bits are 0.

1.3.2 Functions

Then define:

Table 1.3: Bitweise Operatoren und Zuweisungen in C

Operator	C-Syntax	Assoziativität
Bitweise Negation	$\sim$	von rechts nach links
Bitweiser Shift	$\ll, \gg$	von links nach rechts
Bitweises Und	$\&$	von links nach rechts
Bitweises XOR	$\wedge$	von links nach rechts
Bitweises Oder	$ $	von links nach rechts
Zuweisung	$=, \&=, \wedge=, =, \ll=, \gg=$	von rechts nach links

Definition 1.3.2.1

- Set i Bit to 0:

$$w \&= \sim (1 \ll i)$$

- Set i Bit to 1:

$$w |= 1 \ll i$$

- Bit Flip of i :

$$w \wedge= 1 \ll i$$

- Testing i bit is 0 or 1

$$w \& (1 \ll i) \quad \text{equivalent to} \quad (w \gg i) \& 1$$

1.4 Natural Numbers

1.4.1 Construction of Natural Number

We construct the natural number through bytes: Define the function

$$s_n(w) := \begin{cases} (01) & w = (w_0) = (0), n = 1 \\ (10) & w = (w_0) = (1), n = 1 \\ (w_{n-1} \wedge w'_{n-1}, w_{n-1} \oplus w'_{n-1}, w'_{n-2}, \dots, w'_0) & w' = s_{n-1}(w_{n-2}, \dots, w_0) \end{cases}$$

This actually defines

$$s(-) : \cup \{Seq_n()\}$$

which is actually $s(n) = n + 1$.

♠ **Proposition 1.4.1.1** There exists no n -bit word w with $s_n(w) = 0$.

◦ **Proof ...**

□

This implies, that 0 is the initial object.

♠ **Proposition 1.4.1.2** $s_n(w) := (1, q_{n-1}, \dots, q_0) \Rightarrow q_i = 0$

◦ **Proof ...**

□

♠ **Proposition 1.4.1.3** The followings are equivalent:

$$w = (1, \dots, 1) \Leftrightarrow s_n(w) = (1, 0, \dots, 0)$$

◦ **Proof ...**

□

This gives a correspondence:

$$\{f \in 2^{\mathbb{N}} \mid f \text{ is finite}\} \Leftrightarrow \mathbb{N}$$

♠ **Proposition 1.4.1.4** (Injectivity of Function s_n) s_n is injective.

◦ **Proof ...**

□

1.4.2 Order Relation

Definition 1.4.2.1 Suppose v be m -bit word and w be n -bit word.

★ **Corollary 1.4.2.2** Two words are equivalent,

$$v \sim w \Leftrightarrow [v] = [w]$$

◦ **Proof ...**

□

Definition 1.4.2.3 ...

♠ **Proposition 1.4.2.4** ...

◦ **Proof ...**

□

♠ **Proposition 1.4.2.5** ...

◦ **Proof ...**

□

Definition 1.4.2.6 ...

1.5 Integers of Fixed Byte

1.5.1 b -adic representation

Definition 1.5.1.1 Define the b -adic representation of fixed byte:

$$B(w) = \sum_{i=0}^{n-1} w_i b_i$$

Mostly we have $b_i = b^i$, so $B(w)$ is a

Then, recursively:

♠ **Proposition 1.5.1.2** (Horner-Schema)

$$\sum w_i b_i = b \dots (b(bw_{n-1} + w_{n-2}) + w_{n-3} \dots) + w_0$$

◦ **Proof** Omitted. □

♣ **Example 1.5.1.3** 10-adic:

16-adic:

2-adic:

Define

$$U(w) = \sum_{i=0}^{n-1} w_i 2^i \quad \mathbb{U} = \mathbb{N} \cap [0, 2^n)$$

Then,

$$\begin{aligned} U(w) &= \sum_{i=0}^{n-1} w_i 2^i = \sum_{j=0}^{m-1} \sum_{i=0}^3 w_{4j+i} 2^{4j+i} = \\ &= \sum_{j=0}^{m-1} 2^{4j} \sum_{i=0}^3 w_{4j+i} 2^i \\ &= \sum_{j=0}^{m-1} z_j 16^j \quad z_j = U(w_{4j+3}, \dots, w_{4j}) \end{aligned}$$

1.5.2 Additions

Definition 1.5.2.1 With adic representation as above, we have

$$\begin{aligned} u_n(w) &= \\ v_n(v, w) &= \end{aligned}$$

♠ **Proposition 1.5.2.2**

◦ **Proof** ... □

Definition 1.5.2.3 Suppose v, w be n -bit words, then define the **Two's complement**

$$-_u w := \sim w +_u 1$$

Then, define the operator

$$v -_u w := v +_u (-_u w)$$

♠ **Proposition 1.5.2.4** ...

◦ **Proof** ...

□

★ **Corollary 1.5.2.5** ...

◦ **Proof** ...

□

1.5.3 Modifications

1.5.4 Types in C

1.6 The Natural Numbers

Omitted.

1.7 Complexity

1.7.1 Landau-Symbols

Definition 1.7.1.1 (Landau-Symbol)

$$\begin{aligned} f(x) = \mathcal{O}(g(x)) &\Leftrightarrow \exists c \exists x_0 (c \in \mathbb{R}, x_0 \in \mathbb{R}, \forall x (x \geq x_0 \rightarrow f(x) \leq c g(x))) \\ f(x) = o(g(x)) &\Leftrightarrow \forall c (c > 0 \rightarrow \exists x_0 (x_0 \in \mathbb{R} \wedge \forall x (x \geq x_0 \rightarrow f(x) \leq c g(x)))) \\ f(x) = \Theta(g(x)) &\Leftrightarrow \end{aligned}$$

Some of elementary results:

1.7.2 C of Algorithms

1.8 Error Analysis

Chapter 2 Solution of Linear System of Equations

2.1 Matrices, Vectors and their Implementation

2.2 Gauss Algorithm, and LR-Decomposition

2.2.1 Gauss Algorithms

♠ **Proposition 2.2.1.1** Consider $n \times n$ matrix. Then, Gauss-algorithm has the time complexity of

$$\mathcal{O}(n^3)$$

◦ **Proof ...**

□

2.2.2 LR-Decomposition

Given a $A \in M_n(F)$. During the Gauss-elimination, define:

$$l_{ik} := \frac{a_{ik}^{(k-1)}}{a_k^{(k-1)}}$$

Define

$$[G_k]_{ij} := \begin{cases} l_{ik} & j = k, ik + 1, \dots, n \\ 0 & \text{sonst} \end{cases}$$

Then, we have

$$\begin{aligned} A^{(k)} &= (I - G_k) \cdot A^{(k-1)} \\ b^{(k)} &= (I - G_k) \cdot b^{(k-1)} \end{aligned}$$

By

$$[G_k A^{(k-1)}]_{ij} = \sum_{m=1}^n [G_k]_{im} [A^{(k-1)}]_{mj} = [G_k]_{ik} [A^{(k-1)}]_{kj}$$

♠ **Proposition 2.2.2.1** Suppose $A \in M_n(F)$, and Gauss-Elimination can be applied without row exchange.

- $(I - G_{n-1}) \cdot \dots \cdot (I - G_1)A = R;$
- $G_k \cdot G_l = 0, k \leq l;$
- $(I - G_k)^{-1} = (I + G_k);$
- $((I - G_{n-1}) \cdot \dots \cdot (I - G_1))^{-1} = I + \sum G_k = L.$

◦ **Proof ...**

□

★ **Corollary 2.2.2.2**

$$A = L \cdot R$$

◦ **Proof ...**

□

★ **Corollary 2.2.2.3**

$$PA = LR$$

◦ **Proof ...**

□

2.2.3 Pivotelement

2.3 Cholesky-Decomposition

Consider the positive definite matrix.

♠ **Proposition 2.3.0.1**

-
-

◦ **Proof ...**

□

♠ **Proposition 2.3.0.2** Orthonormal Decomposition for s.p.d)

$$A = LL^T$$

◦ **Proof**

-
- $\Rightarrow$ Consider the s.p.d matrix

$$\begin{pmatrix} A_{n-1} & b \\ b^T & a_{nn} \end{pmatrix} \quad b \in \mathbb{R}^{n-1}, A_{n-1} \in M_{n-1}(\mathbb{R})$$

By induction we have $A_{n-1} = L_{n-1} L_{n-1}^T$. Then, set

$$L = \begin{pmatrix} L_{n-1} & 0 \\ c^T & \alpha \end{pmatrix}$$

We have

$$LL^T = \begin{pmatrix} L_{n-1} & 0 \\ c^T & \alpha \end{pmatrix} \begin{pmatrix} L_{n-1}^T & c \\ 0 & \alpha \end{pmatrix} = \begin{pmatrix} A_{n-1} & L_{n-1}^T c \\ c^T L_{n-1} & c^T c + \alpha^2 \end{pmatrix}$$

We just need to get the value:

$$\begin{cases} L_{n-1}^T c = b \\ \alpha = \sqrt{a_{nn} - c^T c} \end{cases}$$

And L_{n-1} is invertible.

□

And we could apply the following correspondence by

$$A = LL^T$$

Finally, we have

$$\sum_{j=1}^n (n-j+1) = \mathcal{O}(n^2) + \frac{1}{6}n^3$$

Chapter 3 Sorting Algorithms

3.1 Definitions

† **Problem 3.1.1** Suppose there exists $n \in \mathbb{N}$ numbers, $z_i \in \mathbb{R}$. Find a permutation $\pi_1, \dots, \pi_n$, with

...

3.2 Bumble Sort

Firstly, $<$ is well-defined. Then, by the applying the following steps:

$w_i > w_{i+1} \rightarrow \text{swap}, i++$
Elsewise $\rightarrow i++$

Until the end. And consider the group of $w_{n-1}, \dots, w_1$. Finally, we get the sequence with

$$w_{n-1} < \dots < w_0$$

3.2.1 Time Complexity

- Worst-Case: $\mathcal{O}(n^2)$;
- Average-Case: $\mathcal{O}(n^2)$;
- Best Basee: $\mathcal{O}(n)$, if we have the extra process

no exchange $\rightarrow \text{stop}$

3.3 Mergesort

The main method:

- Divide the set S into two part S_1, S_2 ;
- Apply the process recursively for S_1, S_3 ;
- Merge;

◇ **Lemma 3.3.0.1** Given $S_x = \{x_1 < \dots < x_m\}$, $S_y = \{y_1 < \dots < y_n\}$. Then $S = S_x \cup S_y$ can be sorted with at most $m + n - 1$ steps.

◦ **Proof** By creating a list

□

3.3.1 Time Complexity

♠ **Proposition 3.3.1.1** The mergesort has the time complexity of $\mathcal{O}(n \log n)$

◦ **Proof** Suppose $n = 2^m$, We have:

$$A(n) = (n - 1) + 2A\left(\frac{n}{2}\right)$$

□

3.4 Quicksort

By applying following process:

- Select Pivot:
- Recursive Call:
- End:

3.4.1 Time complexity

3.5 Heapsort

3.5.1 Binary Tree,Arborescence,Heap

Definition 3.5.1.1 ...

Chapter 4 Graph Algorithms

Consider: Following datas

- n vertices $V = \{v_0, \dots, v_{n-1}\}$;
- m edges $E = \{e_0, \dots, e_{m-1}\}$;

where

$$e_j = \{v_{x_j}, v_{y_j}\} \quad e_j = (v_{x_j}, v_{y_j}) \quad x_j, y_j \in \{0, \dots, n-1\}$$

And define:

$$C_0 := 0 \\ C_{i+1} := C_i + |\delta(v_i)|; i = 0, \dots, n-1$$

Then obviously, we have $C_n = 2m$.

For directed graph, define

$$C_0^+ := 0 \\ C_{i+1}^+ :=$$

Definition 4.0.0.1 The

Chapter 5 Structure of Data

5.1 Elementary Data Structures

5.1.1 Stacks and Queues

5.1.2 Linked Lists

5.1.3 Implementing pointers and objects

5.1.4 Representing Root Trees

5.2 Hash-Table

5.3 Binary Search Trees

5.4 Red-Black Trees

5.5 Augmenting Data Structures

5.6 B-Trees

5.7 Binomial Heaps

5.8 Fibonacci Heaps

5.9 Data Structures for Disjointed Sets